

UADE



Paradigma Orientado a Objetos
Cuatrimestre **1er** de **2026**

TRABAJO PRÁCTICO Y COMPLEMENTARIO DE LA MATERIA
UADE Beats
Módulo 3: Acceso a eventos en vivo

Curso: **Lunes, mañana**

EQUIPO:

1204860	Ferraro, Leandro Martin
1181269	Gabian, Lucia
1185242	Caggia, Nicolás
1215429	Diaz Salla, Juan Ignacio

.....Grupo 9.....

Fecha de entrega: 13/06/2026

Observaciones:

.....

.....

.....

.....

.....

1. Descripción del módulo desarrollado

El módulo correspondiente al equipo consta de la gestión de acceso a eventos en vivo dentro de la plataforma UADE Beats. Este componente forma parte de la arquitectura general del sistema y es responsable de administrar recitales, shows y presentaciones en vivo transmitidas o gestionadas dentro de la plataforma.

El objetivo del módulo es controlar y validar el acceso de usuarios a los eventos, garantizando que el ingreso se realice de acuerdo con las reglas establecidas por el sistema. Para ello, se contempla la gestión del ciclo de vida de los eventos, el control de capacidad máxima, la habilitación de usuarios, la validación de horarios de acceso, la verificación de planes de suscripción compatibles con cada experiencia y el control de disponibilidad según el estado actual del evento.

Asimismo, el módulo incorpora mecanismos de trazabilidad mediante el registro de todos los intentos de acceso, permitiendo almacenar información relacionada tanto con ingresos autorizados como con ingresos rechazados y sus respectivos motivos. Además, se implementa un sistema de validaciones y excepciones específicas ante situaciones inválidas, tales como intentos de ingreso fuera del horario permitido, eventos cancelados, pausados o finalizados, usuarios sin autorización, intentos de acceso duplicados, capacidad agotada o planes de suscripción insuficientes.

Desde el punto de vista del diseño orientado a objetos, el módulo se integra con entidades base de la plataforma como Usuario, Evento, Artista, AccesoEvento, PlanSuscripcion y EstadoEvento, manteniendo una arquitectura modular donde cada clase posee responsabilidades específicas y bien definidas. La coordinación de las operaciones principales del módulo se encuentra centralizada en la clase GestorEventosEnVivo, encargada de administrar usuarios, artistas, eventos, solicitudes de ingreso y registros generados por el sistema.

Además, el desarrollo incorpora mecanismos de persistencia mediante archivos de texto, permitiendo almacenar y recuperar información correspondiente a usuarios, artistas, eventos y registros de acceso. Asimismo, el sistema cuenta con una interfaz gráfica desarrollada en Java Swing que permite gestionar las principales funcionalidades del módulo de manera visual, complementando la ejecución mediante consola.

En consecuencia, el alcance del módulo se centra en la administración y control de acceso a eventos en vivo, incluyendo la gestión de eventos, artistas, usuarios, permisos de acceso, validaciones de reglas de negocio, registro de accesos realizados y persistencia de la información dentro de la plataforma.

2. Interpretación del problema específico y alcance del proyecto

El problema específico que debe resolver el equipo consiste en diseñar una solución modular que permita administrar y controlar el acceso de usuarios a eventos en vivo dentro de la plataforma UADE Beats. No se trata únicamente de representar recitales o shows, sino de desarrollar la lógica necesaria para garantizar que el ingreso de los usuarios se realice bajo las condiciones y restricciones establecidas por el sistema.

Cada vez que un usuario solicita el acceso a un evento, el sistema debe evaluar distintas reglas de negocio antes de autorizar su ingreso. Entre ellas se encuentran la validación de la autorización del usuario, la disponibilidad del evento según su estado actual, el control de la capacidad máxima permitida, la verificación del horario de acceso, la compatibilidad entre el plan de suscripción del usuario y el plan mínimo requerido por el evento y la verificación de que el usuario no se encuentre conectado previamente al mismo recital.

Además, el módulo debe gestionar el ciclo de vida del evento mediante diferentes estados, tales como Programado, En Espera, En Curso, Pausado, Finalizado y Cancelado, permitiendo determinar cuándo un evento se encuentra disponible para recibir nuevos ingresos. Asimismo, los eventos pueden asociarse con uno o varios artistas dentro de la plataforma, manteniendo la relación entre las presentaciones en vivo y sus respectivos participantes o creadores.

Otra responsabilidad del sistema consiste en mantener la trazabilidad de los accesos realizados, registrando tanto los ingresos autorizados como los intentos rechazados junto con sus respectivos motivos. Además, el módulo permite conservar la información de usuarios, artistas, eventos y registros de acceso mediante mecanismos de persistencia, posibilitando recuperar los datos almacenados previamente. Para ello, se aplican validaciones y excepciones específicas ante situaciones como accesos fuera de horario, eventos no disponibles, capacidad agotada, usuarios sin autorización, usuarios ya conectados o planes de suscripción insuficientes.

Desde el punto de vista del diseño orientado a objetos, la solución distribuye sus responsabilidades entre distintas clases colaboradoras. La administración general del módulo se centraliza en `GestorEventosEnVivo`, encargado de coordinar la gestión de usuarios, artistas, eventos, accesos, registros y la persistencia de la información. Las condiciones de autorización y permisos se modelan mediante `AccesoEvento`, que representa la relación entre un usuario y un evento validando los requisitos previos de ingreso. Las características generales y el ciclo de vida de los eventos se gestionan mediante `Evento`, las reglas específicas de acceso y conexión de usuarios se implementan en `RecitalEnVivo` y la trazabilidad de los ingresos se

almacena mediante RegistroAcceso, evitando concentrar toda la lógica del negocio en una única clase.

3. Propuesta preliminar de clases principales del módulo

Para el desarrollo del módulo de acceso a eventos en vivo se propone una estructura orientada a objetos integrada con la estructura base común definida para la plataforma UADE Beats. El módulo utiliza principalmente las clases Evento, Usuario y AccesoEvento, junto con las enumeraciones EstadoEvento y PlanSuscripcion. Con esto, incorporamos nuevas responsabilidades vinculadas al control de acceso, capacidad, validaciones y trazabilidad de eventos en vivo.

- Evento: clase abstracta que ilustra cualquier experiencia en vivo dentro de la plataforma. Es la base obligatoria de todos los eventos específicos.

Atributos:

- id: int
- titulo: String
- descripcion: String
- fechaHoraInicio: LocalDateTime
- fechaHoraFin: LocalDateTime
- estado: EstadoEvento
- capacidadMaxima: int
- artistas: List<Artista>

Métodos:

- +iniciarEvento(): void
- +finalizarEvento(): void
- +cancelarEvento(): void
- +pausarEvento(): void
- +reanudarEvento(): void
- +mostrarEstado(): String
- +estaDentroDelHorario(momento: LocalDateTime): boolean
- +agregarArtista(artista: Artista): void

- Usuario: clase que representa a cualquier usuario registrado de la plataforma.

Atributos:

- id: int
- nombreUsuario: String
- nombre: String
- apellido: String
- email: String
- contrasena: String
- planSuscripcion: PlanSuscripcion
- activo: boolean
- sesionIniciada: boolean

Métodos:

+iniciarSesion(): void
+cerrarSesion(): void
+actualizarPerfil(): void

- RecitalEnVivo: clase concreta derivada de Evento que representa recitales, shows o presentaciones en vivo transmitidas desde la plataforma.

Atributos heredados:

-id: int
-titulo: String
-descripcion: String
-fechaHoraInicio: LocalDateTime
-fechaHoraFin: LocalDateTime
-estado: EstadoEvento
-capacidadMaxima: int

Atributos propios:

-planMinimoRequerido: PlanSuscripcion
-ubicacion: String
-esStreaming: boolean
-exclusivo: boolean
-usuariosConectados: Set<Usuario>

Métodos heredados:

+iniciarEvento(): void
+finalizarEvento(): void
+mostrarEstado(): String

Métodos propios:

+verificarCapacidad(): boolean
+permitirIngreso(usuario: Usuario): void
+validarDisponibilidad(momento: LocalDateTime): void
+expulsarUsuario(usuario: Usuario): void

- Artista: clase que representa a los artistas o bandas que participan en eventos dentro de la plataforma.

Atributos:

-id: int
-nombreArtistico: String
-generoPrincipal: String
-biografia: String
-verificado: boolean
-eventosCreados: List<Evento>
-obras: List<String>
-redesSociales: Map<String, String>

Métodos:

+crearEvento(evento: Evento): void
+actualizarInformacionArtistica(String nuevaBiografia): String
+agregarObra(titulo: String): void
+agregarRedSocial(red: String, url: String) : void

- AccesoEvento: clase utilizada para representar la relación entre un usuario y un evento al que desea acceder, registrando permisos y habilitaciones de ingreso.

Atributos:

-id: int
-usuario: Usuario
-evento: Evento
-fechaRegistro: LocalDateTime
-habilitado: boolean
-tipoAcceso: String
-planRequerido: PlanSuscripcion
-accesoPrioritario: boolean

Métodos:

+validarIngreso(): boolean
+validarAutorizacion(): void
+validarPlan(): void
+validarPlanUsuario(): boolean
+generarAcceso(): void
+cancelarAcceso(): void

- GestorEventosEnVivo: clase encargada de administrar los eventos en vivo disponibles dentro del módulo y coordinar operaciones generales relacionadas con su gestión.

Atributos:

-eventos: List<Evento>
-registros: List<RegistroAcceso>
-usuariosRegistrados: List<Usuario>
-artistasRegistrados: List<Artista>
-siguienteIdAcceso: int

Métodos:

+cancelarEvento(evento: Evento): void
+buscarEventoPorId(id: int): Evento
+buscarUsuarioPorId(id: int): Usuario
+listarEventosDisponibles(): List<Evento>
+listarEventos(): List<Evento>
+listarUsuarios(): List<Usuario>
+listarRegistros(): List<RegistroAcceso>
+registrarUsuario(usuario: Usuario): void
+crearEvento(evento: RecitalEnVivo): void
+registrarAcceso(registro: RegistroAcceso): void

+solicitarIngreso(idUsuario: int, idEvento: int): RegistroAcceso
+expulsarUsuario(idEvento: int, idUsuario: int): boolean

- RegistroAcceso: clase responsable de mantener trazabilidad sobre ingresos e intentos de acceso realizados por los usuarios.

Atributos:

-usuario: Usuario
-evento: Evento
-fechaHoraIngreso: LocalDateTime
-fechaHoraSalida: LocalDateTime
-exitoso: boolean
-motivoRechazo: String

Métodos:

+generarRegistro(): RegistroAcceso
+guardarRegistro(): void
+registrarIntentoFallido(): RegistroAcceso
+registrarSalida(): void

- PlanSuscripcion: enumeración que representa los planes de suscripción a los que tiene acceso el usuario.

FREE
PREMIUM
ARTIST_PASS

- EstadoEvento:

PROGRAMADO
EN_CURSO
EN_ESPERA
PAUSADO
FINALIZADO
CANCELADO

En conclusión:

El módulo contempla validaciones relacionadas con:

- acceso fuera de horario;
- capacidad agotada;
- usuario sin autorización;
- plan de suscripción insuficiente;
- eventos cancelados, pausados o finalizados;
- usuario ya conectado;

Estas validaciones se implementan mediante lógica de negocio distribuida entre las clases correspondientes y manejo de excepciones específicas del módulo, evitando concentrar demasiadas responsabilidades en una única entidad.

4. Relaciones de las clases principales con la estructura base común

Podemos conectar las clases mencionadas anteriormente con la estructura base común estableciendo diversas relaciones:

Herencia

RecitalEnVivo ----|> Evento: RecitalEnVivo es una especialización de Evento, ya que representa un tipo específico de experiencia en vivo dentro de la plataforma. Reutiliza los atributos y comportamientos generales asociados a los eventos y agrega características particulares relacionadas con el acceso y la interacción de los usuarios.

Asociación

Usuario (1) ↔ AccesoEvento (0..*): AccesoEvento se relaciona con Usuario y Evento representando la autorización o habilitación de ingreso de un usuario a un recital o presentación en vivo. Un usuario puede tener múltiples accesos asociados a diferentes eventos y cada acceso corresponde a un único usuario.

Evento (1) ↔ AccesoEvento (0..*): Cada acceso se encuentra asociado a un evento específico, mientras que un evento puede tener múltiples solicitudes o autorizaciones de acceso asociadas.

Usuario (1) ↔ RegistroAcceso (0..*): Un usuario puede poseer varios registros de acceso relacionados con su actividad dentro de los eventos.

Evento (1) ↔ RegistroAcceso (0..*): Un evento puede poseer múltiples registros asociados a intentos de acceso realizados por distintos usuarios.

Evento (0..*) ↔ Artista (0..*): Un evento puede contar con la participación de uno o varios artistas y un artista puede participar en distintos eventos dentro de la plataforma.

RecitalEnVivo (1) ↔ Usuario (0..*): Un recital mantiene una colección de usuarios conectados durante la transmisión, permitiendo controlar el acceso y evitando usuarios duplicados mediante una estructura de tipo Set.

Usuario (1) ↔ PlanSuscripcion (1): Cada usuario posee un único plan de suscripción que determina sus permisos y nivel de acceso dentro de la plataforma.

PlanSuscripcion (1) ↔ RecitalEnVivo (0..*): Un mismo tipo de plan puede habilitar el acceso a múltiples recitales, mientras que cada recital posee un único plan mínimo requerido.

Agregación

GestorEventosEnVivo (1) ◇—— Evento (0..*): El gestor administra los eventos disponibles dentro del módulo, aunque estos pueden existir independientemente del gestor.

GestorEventosEnVivo (1) ◇—— Artistas (0..*): El gestor administra a los artistas relacionados con los eventos, pero estos pueden existir independientemente de él.

GestorEventosEnVivo (1) ◇—— Usuario (0..*): El gestor administra los usuarios registrados dentro del módulo, aunque los usuarios pueden existir independientemente de este.

5. Breve descripción de la funcionalidad implementada

El módulo de gestión de acceso a eventos se encarga de administrar las autorizaciones, validaciones y registro de ingreso de usuarios a eventos en vivo, garantizando el cumplimiento de las reglas establecidas por el sistema.

Gestión de accesos

En esta sección se definen las funcionalidades relacionadas con el control de permisos de acceso de los usuarios a los distintos eventos.

→ El sistema permite validar y gestionar las solicitudes de ingreso de los usuarios a los eventos.

→ El sistema permite autorizar o rechazar el acceso según el cumplimiento de las condiciones establecidas.

Verificación

En esta etapa, el sistema evalúa si se cumplen todas las condiciones necesarias para permitir el acceso del usuario al evento.

→ El sistema verifica si el usuario posee autorización para ingresar al evento.

→ El sistema verifica si el usuario cuenta con el plan de suscripción necesario para participar del evento.

→ El sistema verifica si el evento se encuentra disponible según su estado actual.

- El sistema verifica si el evento no alcanzó su capacidad máxima.
- El sistema verifica si el usuario ingresa dentro de un horario válido.
- El sistema verifica que el usuario no se encuentre conectado previamente al mismo evento.

Acceso

Una vez validadas las condiciones, el sistema gestiona el ingreso del usuario al evento y controla su permanencia dentro del mismo.

- El sistema registra los accesos autorizados de los usuarios al evento.
- El sistema registra los intentos de acceso inválidos o rechazados, almacenando el motivo correspondiente.
- El sistema permite expulsar a un usuario conectado al evento cuando sea necesario.

Manejo de Eventos

Esta sección contempla las funcionalidades necesarias para la administración de los eventos dentro del sistema.

- El sistema permite crear eventos de transmisión en vivo.
- El sistema permite iniciar, pausar, reanudar, finalizar y cancelar eventos en vivo.
- El sistema permite buscar eventos a través de su ID.
- El sistema permite listar los eventos disponibles y los eventos registrados en el sistema.
- El sistema permite asociar artistas a los eventos en vivo y mantener la relación entre los artistas y sus presentaciones.

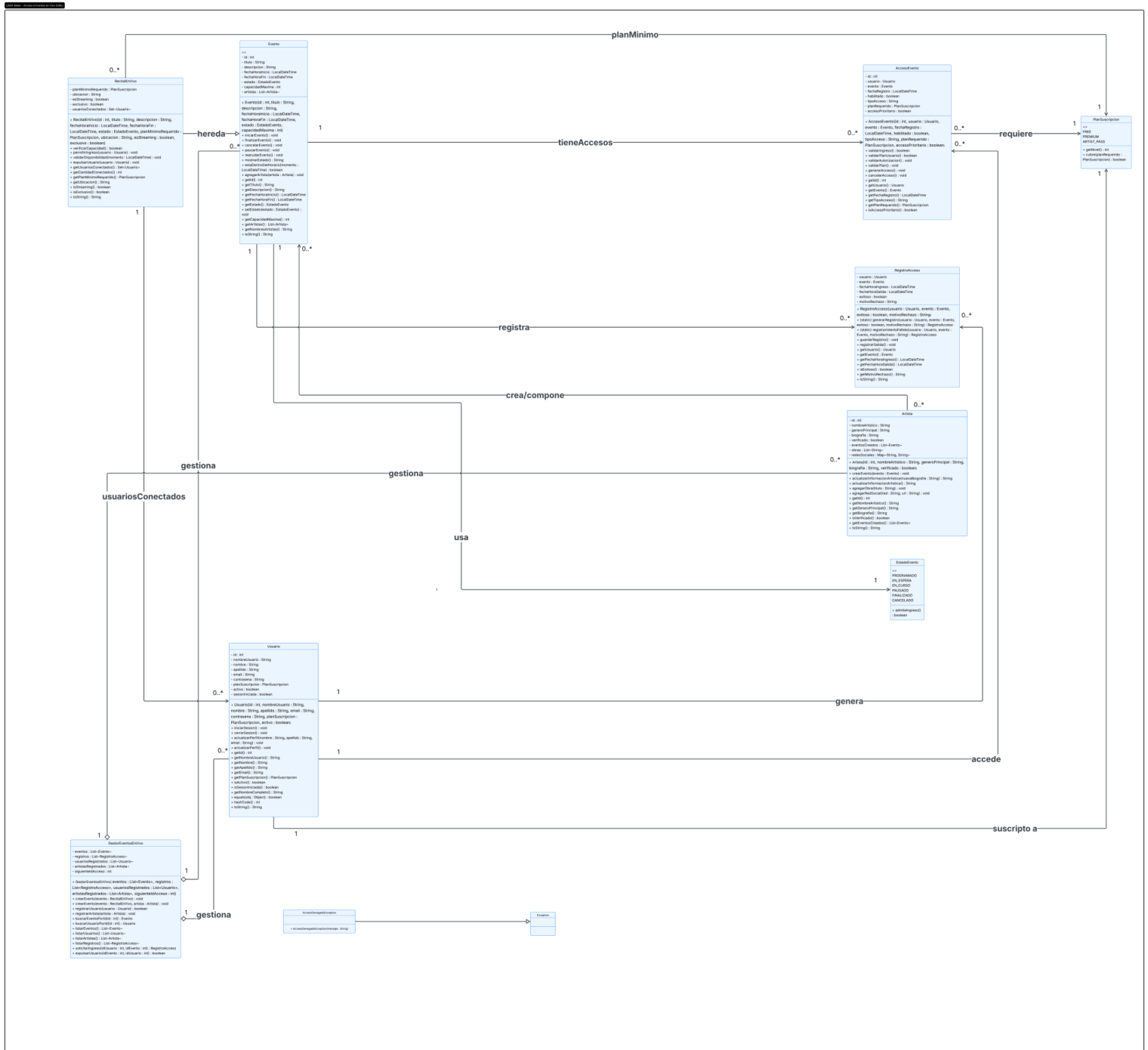
Manejo de errores

El sistema gestiona situaciones en las que no se cumplen las condiciones requeridas o se producen fallos durante la operación.

Paradigma orientado a objetos

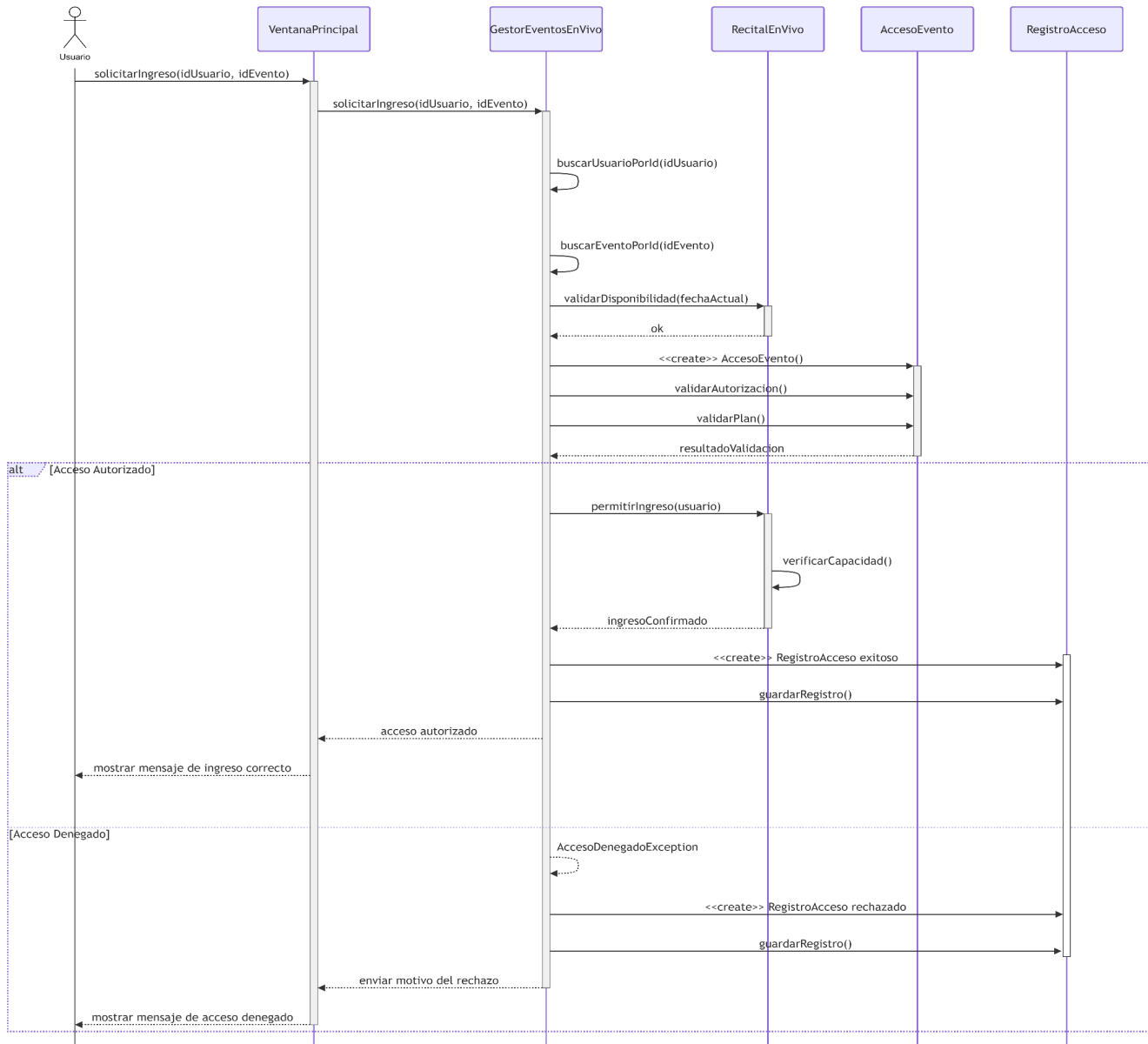
- El sistema responde adecuadamente ante intentos de acceso inválidos, denegando el ingreso cuando no se cumplen las condiciones establecidas.
- El sistema gestiona errores mediante validaciones y excepciones específicas que indican la causa del rechazo.
- El sistema contempla errores relacionados con:
 - acceso fuera de horario;
 - capacidad agotada;
 - eventos cancelados, pausados o finalizados;
 - usuarios sin autorización;
 - planes de suscripción insuficientes;
 - intentos de ingreso de usuarios ya conectados.

6. Esquema del Diagrama de Clases

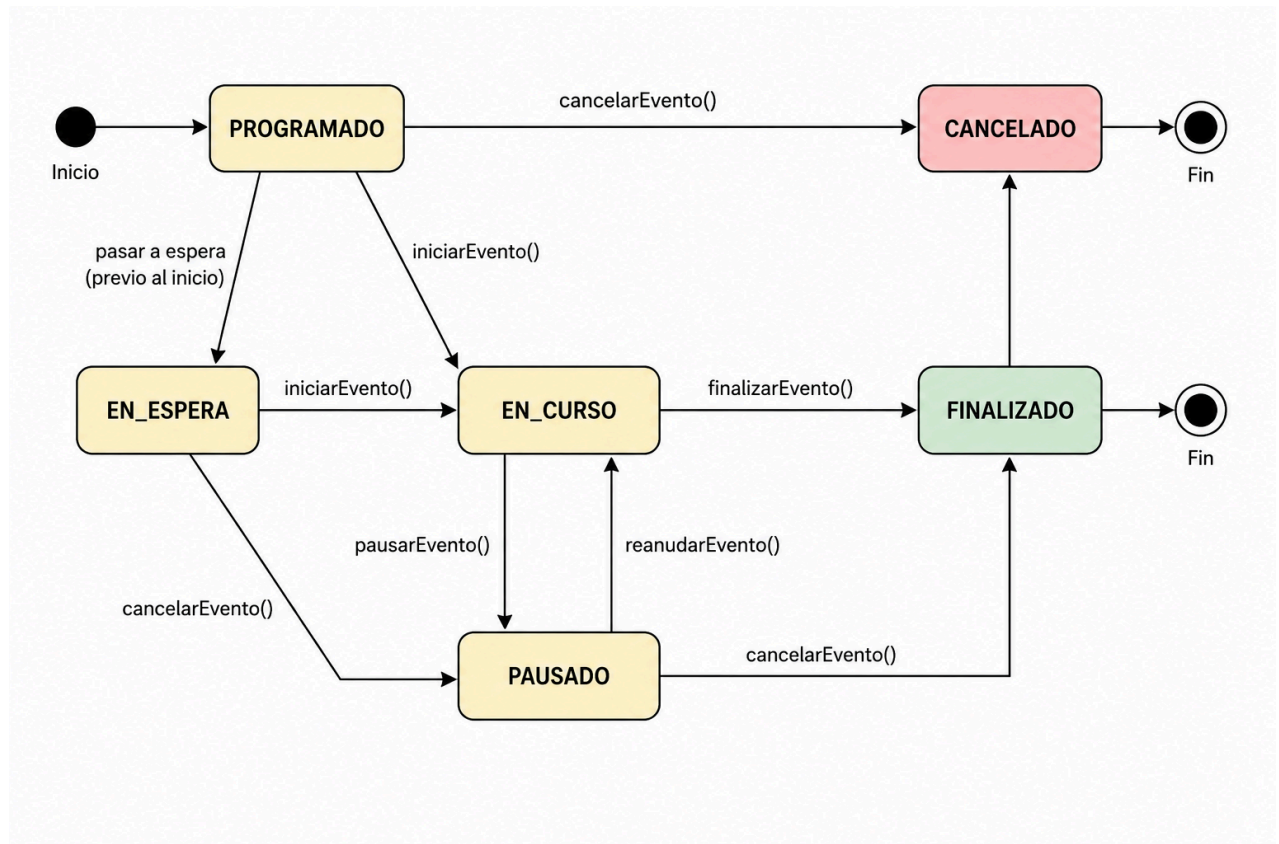


7. Diagrama de Secuencia de una funcionalidad relevante

La funcionalidad que se tuvo en cuenta al momento de hacer el diagrama fue Validación de reglas de negocio y Autorización de ingreso a recitales en vivo. Esta funcionalidad representa el núcleo operativo del Módulo. Modela el proceso sincrónico y secuencial que se desencadena cuando un miembro del equipo de la plataforma solicita el acceso a una transmisión o evento en tiempo real. Su relevancia radica en que coordina la comunicación de múltiples entidades del dominio para evaluar de forma robusta e integral criterios comerciales, lógicos y operativos antes de conceder el acceso.



8. Diagrama de Estado



9. Descripción de las principales decisiones de diseño adoptadas

Durante el desarrollo del módulo asignado adoptamos diversas decisiones de diseño orientadas a preservar la coherencia con la estructura base de UADE Beats y aplicar correctamente los principios del paradigma orientado a objetos.

Decidimos que RecitalEnVivo sea una subclase de Evento. Al estructurar Evento como una clase abstracta, logramos centralizar atributos y comportamientos generales relacionados con el ciclo de vida de los eventos, tales como los horarios, la capacidad máxima, el estado y el identificador del evento, permitiendo reutilizar código y evitar duplicación innecesaria entre los distintos tipos de eventos.

Para manejar el ciclo de vida de los recitales de forma segura, implementamos la enumeración EstadoEvento, la cual nos permitió restringir los estados posibles a un conjunto fijo y determinado (PROGRAMADO, EN_ESPERA, EN_CURSO, PAUSADO, FINALIZADO y CANCELADO), evitando que el sistema procese valores inválidos o inconsistentes al momento de administrar y validar los eventos.

Por otro lado, AccesoEvento representa la autorización que vincula a un usuario con un evento determinado y concentra las validaciones previas al ingreso, como la

habilitación del usuario y la compatibilidad entre el plan de suscripción contratado y el plan requerido por el evento. La incorporación de atributos como `planRequerido`, `tipoAcceso` y `accesoPrioritario` responde a la necesidad de almacenar condiciones específicas de cada autorización, permitiendo que el sistema pueda gestionar diferentes tipos de acceso sin sobrecargar a las entidades `Usuario` o `Evento` con información particular de cada solicitud de ingreso. De esta manera, se mantiene una mejor separación de responsabilidades y se favorece la extensibilidad del modelo ante posibles tipos de acceso futuros.

Asimismo, `RegistroAcceso` se encarga de mantener la trazabilidad de ingresos autorizados y rechazados, almacenando información relevante sobre cada intento de acceso y su resultado. Por su parte, `GestorEventosEnVivo` cumple el rol de clase coordinadora del módulo, administrando eventos, usuarios registrados, registros de acceso y centralizando el flujo principal de solicitudes de ingreso.

Además, incorporamos la relación entre `Evento` y `Artista` para representar los artistas participantes de cada presentación en vivo, permitiendo mantener una mayor integración con la estructura general de UADE Beats.

Para el manejo de situaciones inválidas, implementamos una excepción específica denominada `AccesoDenegadoException`, la cual permite identificar claramente los distintos motivos por los cuales un ingreso puede ser rechazado, como falta de autorización, capacidad agotada, eventos no disponibles, planes de suscripción insuficientes o intentos de ingreso duplicados.

A su vez, utilizamos colecciones genéricas de Java como `List`, `Set` y `Map` para administrar dinámicamente conjuntos de objetos relacionados con eventos, usuarios, artistas y registros del sistema. En particular, la utilización de `Set` en los usuarios conectados permite evitar la existencia de usuarios duplicados dentro de un mismo recital, mientras que `Map` se utiliza en `Artista` para organizar sus redes sociales mediante pares clave-valor.

Por último, el diseño contempla el principio de encapsulamiento mediante atributos privados y acceso controlado a través de métodos públicos. También se incorporaron conceptos como enumeraciones, herencia, abstracción y excepciones propias, favoreciendo una implementación segura, mantenible y alineada con los pilares del paradigma orientado a objetos trabajados a lo largo de la cursada.

10. Evidencia de uso de herencia, encapsulamiento, interfaces o clases abstractas cuando corresponda;

Se adjuntan las siguientes evidencias:

- **Uso de herencia**

Se implementó herencia mediante la clase RecitalEnVivo, la cual extiende a la clase abstracta Evento. Esta relación permite reutilizar los atributos y comportamientos generales asociados a cualquier evento dentro de la plataforma, como el identificador, título, descripción, horarios, estado, capacidad máxima y gestión de artistas.

Ejemplo implementado:

```
* - validacion de estado y horario.
*/
public class RecitalEnVivo extends Evento {

    // Plan minimo que necesita un usuario para entrar.
    private PlanSuscripcion planMinimoRequerido; 3 usages

    // ...

    // Constructor completo del recital.
    public RecitalEnVivo(int id,
        String titulo,
        String descripcion,
        LocalDateTime fechaHoraInicio,
        LocalDateTime fechaHoraFin,
        EstadoEvento estado,
        int capacidadMaxima,
        PlanSuscripcion planMinimoRequerido,
        String ubicacion,
        boolean esStreaming,
        boolean exclusivo) {

        super(id,
            titulo,
            descripcion,
            fechaHoraInicio,
            fechaHoraFin,
            estado,
            capacidadMaxima);
```

Esto demuestra que RecitalenVivo reutiliza los atributos y la inicialización definidos en Evento

- **Uso de clases abstractas**

La clase evento fue desarrollada como una clase abstracta, utilizada como base común para los distintos tipos de eventos dentro de la plataforma UADE Beats. Esta clase centraliza atributos y comportamientos generales de cualquier evento, como su identificación, información descriptiva, horarios, estado, capacidad máxima y gestión de artistas. Al declararse como abstracta, se evita la creación directa de objetos de tipo Evento, estableciendo una estructura base que debe ser extendida por clases más específicas como RecitalEnVivo

Ejemplo implementado:

```
/*
 * Clase abstracta base para cualquier evento de UADE Beats.
 *
 * Contiene los datos comunes de un evento y define operaciones
 * generales del ciclo de vida. RecitalEnVivo hereda de esta clase.
 */
public abstract class Evento {

    // Identificador unico del evento.
    private int id; 3 usages

    // Titulo visible del evento.
    private String titulo; 3 usages
```

- **Encapsulamiento**

Las clases implementadas utilizan el principio de encapsulamiento mediante la declaración de atributos privados y el acceso controlado a los mismos a través de métodos públicos, principalmente getters y métodos de negocio. Esto permite proteger el estado interno de los objetos y evitar modificaciones directas desde otras clases del sistema.

Por ejemplo, en la clase **Usuario**

```
public class Usuario {  
  
    // Identificador único del usuario.  
    private int id;  
  
    // Nombre de cuenta dentro de la plataforma.  
    private String nombreUsuario;  
  
    // Nombre real.  
    private String nombre;  
  
    // Apellido real.  
    private String apellido;  
  
    // Email de contacto.  
    private String email;  
  
    // Contraseña simplificada para el prototipo.  
    private String contraseña;  
  
    // Plan contratado por el usuario.  
    private PlanSuscripcion planSuscripcion;  
  
    // Indica si el usuario está habilitado en la plataforma.  
    private boolean activo;  
  
    // Indica si inicio sesión.  
    private boolean sesionIniciada;  
  
  
    // Getters de consulta.  
    public int getId() { return id; }  
  
    public String getNombreUsuario() { return nombreUsuario; }  
  
    public String getNombre() { return nombre; }  
  
    public String getApellido() { return apellido; }  
  
    public String getEmail() { return email; }  
  
    public PlanSuscripcion getPlanSuscripcion() { return planSuscripcion; }  
  
    public boolean isActive() { return activo; }  
}
```

con sus respectivos getters.

- **Uso de enumeraciones**

Se implementaron enumeraciones (enum) para representar conjuntos de valores fijos dentro del sistema, evitando el uso de cadenas de texto y reduciendo la posibilidad de asignar valores inválidos. En el módulo se utilizan las enumeraciones EstadoEvento y PlanSuscripcion, las cuales permiten modelar los distintos estados

del ciclo de vida de un evento y los niveles de acceso disponibles para los usuarios dentro de la plataforma. Por ejemplo:

```
public enum EstadoEvento {  
  
    // Evento creado, pero todavía no disponible para ingresar.  
    PROGRAMADO,  
  
    // Evento preparado o en sala de espera previa.  
    EN_ESPERA,  
  
    // Unico estado en el que se permite el ingreso de usuarios.  
    EN_CURSO,  
  
    // Evento detenido temporalmente.  
    PAUSADO,  
  
    // Evento terminado.  
    FINALIZADO,  
  
    // Evento cancelado.  
    CANCELADO;  
  
    // Regla de negocio: solo se puede entrar si el evento esta en curso.  
    public boolean admiteIngreso() { return this == EN_CURSO; }  
}
```

Esto permite representar valores constantes de manera más segura y organizada.

- **Uso de excepciones personalizadas**

Se implementó la clase AccesoDenegadoException, utilizada para representar errores específicos relacionados con el acceso de usuarios a eventos en vivo. Esta excepción permite identificar situaciones en las que un ingreso debe ser rechazado, como usuarios sin autorización, planes de suscripción insuficientes, eventos no disponibles, capacidad agotada o intentos de acceso duplicados, permitiendo un manejo más claro y organizado de los errores dentro del sistema.

Ejemplo implementado:

```
*/  
public class AccesoDenegadoException extends Exception {  
  
    // Recibe el motivo concreto del rechazo y lo guarda como mensaje.  
    public AccesoDenegadoException(String mensaje) { super(mensaje); }  
}
```

11. Uso inicial de colecciones, validaciones y manejo de errores si el módulo lo requiere

Dentro de la implementación actual están incorporados el uso de colecciones, validaciones y manejo de errores para administrar correctamente el acceso a recitales y eventos en vivo.

- **Uso de colecciones**

Se utilizaron colecciones de Java como List, ArrayList, Set, HashSet, Map y HashMap para administrar dinámicamente los distintos elementos del sistema. En la clase RecitalEnVivo se implementó un Set de usuarios mediante un HashSet para almacenar los usuarios conectados, evitando duplicados y garantizando que un mismo usuario no pueda ingresar más de una vez al mismo evento.

```
// Set de usuarios conectados. Se usa Set para evitar duplicados.  
private Set<Usuario> usuariosConectados; 7 usages
```

La colección de usuarios conectados se inicializa dentro del constructor de RecitalEnVivo, asegurando que cada recital tenga su propio conjunto de usuarios conectados desde el momento de su creación.

```
  
    this.usuariosConectados = new HashSet<>();  
}
```

Además, en la clase Artista se utiliza una estructura Map para administrar las redes sociales del artista mediante una relación clave-valor, donde se almacena el nombre de la red social junto con su correspondiente enlace.

```
// Redes sociales: clave = nombre de red, valor = URL.  
private Map<String, String> redesSociales; 2 usages
```

La estructura de redes sociales se inicializa en el constructor de la clase Artista para garantizar que cada artista posea su propia colección de redes desde su creación.

```
    this.redesSociales = new HashMap<>();  
}
```

- **Validaciones implementadas**

El sistema incorpora distintas validaciones relacionadas con el ingreso de usuarios a los eventos en vivo, garantizando que se cumplan las reglas de negocio establecidas por la plataforma.

En la clase RecitalEnVivo se implementaron validaciones relacionadas con la capacidad disponible del evento, el estado del recital, el horario permitido de ingreso y la verificación de usuarios previamente conectados.

ejemplo :

```
// Devuelve true si todavía hay lugar disponible.  
public boolean verificarCapacidad() { return usuariosConectados.size() < getCapacidadMaxima(); }  
  
// Regla central: solo EN_CURSO admite ingreso.  
if (!getEstado().admiteIngreso()) {  
    throw new AccesoDenegadoException(  
        "Evento no disponible: debe estar EN_CURSO."  
    );  
}
```

En la clase AccesoEvento se implementaron validaciones relacionadas con la autorización del usuario y la compatibilidad entre el plan contratado y el plan requerido para el acceso al evento.

```
// Lanza excepcion si el usuario no está habilitado.
public void validarAutorizacion() throws AccesoDenegadoException {
    if (!habilitado || !usuario.isActivo()) {
        throw new AccesoDenegadoException(
            "Usuario sin autorizacion para ingresar."
        );
    }
}

// Lanza excepcion si el plan del usuario es insuficiente.
public void validarPlan() throws AccesoDenegadoException {
    if (!validarPlanUsuario()) {
        throw new AccesoDenegadoException(
            "Plan insuficiente. Requiere "
            + planRequerido
            + " y el usuario posee "
            + usuario.getPlanSuscripcion()
            + "."
        );
    }
}
}
```

- **Manejo de errores y excepciones**

Se implementó una excepción personalizada denominada `AccesoDenegadoException`, utilizada para representar situaciones inválidas relacionadas con el acceso de usuarios a los eventos dentro del módulo, como usuarios sin autorización, planes de suscripción insuficientes, eventos no disponibles, capacidad agotada o intentos de ingreso duplicados. En la clase `RecitalEnVivo` se utiliza la excepción mediante las sentencias `throw` y `throws` para informar errores cuando no se cumplen las condiciones necesarias para permitir el ingreso. Un ejemplo de ello es la validación de usuarios previamente conectados, donde se utiliza el método `contains()` sobre un `Set` de usuarios, aprovechando la implementación de `equals()` en la clase `Usuario` para detectar duplicados y lanzar la excepción correspondiente.

```
public void permitirIngreso(Usuario usuario) throws AccesoDenegadoException {  
    // Primero se valida estado y horario.  
    validarDisponibilidad(LocalDateTime.now());  
  
    // Si no hay capacidad, se rechaza el ingreso.  
    if (!VerificarCapacidad()) {  
        throw new AccesoDenegadoException(  
            "Capacidad agotada."  
        );  
    }  
  
    // El Set usa equals/hashCode de Usuario para detectar duplicados.  
    if (usuariosConectados.contains(usuario)) {  
        throw new AccesoDenegadoException(  
            "El usuario ya se encuentra conectado."  
        );  
    }  
  
    // Si todas las validaciones pasan, el usuario queda conectado.  
    usuariosConectados.add(usuario);  
}
```

También se implementaron validaciones de estado y disponibilidad del evento utilizando la misma excepción.

```
// Regla central: solo EN_CURSO admite ingreso.  
if (!getEstado().admiteIngreso()) {  
    throw new AccesoDenegadoException(  
        "Evento no disponible: debe estar EN_CURSO."  
    );  
}
```

Además se implementó manejo de errores utilizando bloques try-catch en la clase **App**, permitiendo capturar excepciones sin detener la ejecución del programa y mostrando un mensaje adecuado al usuario.

```
try {
    // Ejecuta la accion recibida por parametro.
    if ("iniciar".equals(accion)) {
        evento.iniciarEvento();
        System.out.println("Evento iniciado.");
    } else if ("pausar".equals(accion)) {
        evento.pausarEvento();
        System.out.println("Evento pausado.");
    } else if ("reanudar".equals(accion)) {
        evento.reanudarEvento();
        System.out.println("Evento reanudado.");
    } else if ("finalizar".equals(accion)) {
        evento.finalizarEvento();
        System.out.println("Evento finalizado.");
    }
} catch (AccesoDenegadoException e) {
    System.out.println("No se pudo cambiar el estado: "
        + e.getMessage());
}
```

12. Evidencias de pruebas funcionales y ejecución

Para realizar las pruebas funcionales del módulo, se ejecuta la clase principal **App**, la cual inicia una interfaz por consola con un menú de opciones que permite interactuar con el sistema y probar los distintos casos de uso implementados.

El flujo de prueba realizado es el siguiente:

Opción 1 – Registrar usuario

Permite crear nuevos usuarios dentro de la plataforma, ingresando sus datos personales y seleccionando el plan de suscripción correspondiente (**FREE**, **PREMIUM** o **ARTIST_PASS**). Esto permite realizar posteriormente pruebas de acceso con distintos niveles de permisos.

Opción 2 – Registrar artista

Permite registrar nuevos artistas indicando su nombre artístico, género musical, biografía y estado de verificación. Los artistas creados podrán asociarse posteriormente a nuevos eventos en vivo.

Opción 3 – Crear evento manualmente

Permite crear un nuevo recital en vivo seleccionando previamente un artista registrado. Durante la creación se definen datos como título, descripción, fecha y horario, capacidad máxima, plan mínimo requerido, ubicación y características del evento.

Opción 4 – Ver usuarios registrados

Permite visualizar todos los usuarios disponibles dentro del sistema y verificar que la información haya sido almacenada correctamente.

Opción 5 – Ver artistas registrados

Permite visualizar los artistas cargados dentro del sistema y comprobar su correcta asociación con la plataforma.

Opción 6 – Ver eventos administrados

Permite consultar todos los eventos registrados, mostrando sus datos generales, estado actual, capacidad y artistas asociados.

Opción 7 – Iniciar evento

Permite modificar el estado de un evento que se encuentre en **PROGRAMADO** o **EN_ESPERA**, cambiándolo a **EN_CURSO**, habilitando así la posibilidad de que los usuarios puedan solicitar el ingreso.

Opción 8 – Solicitar ingreso a evento

Representa el caso de uso principal del módulo. El sistema solicita el ID del usuario y del evento, y luego ejecuta todas las validaciones correspondientes: autorización del usuario, compatibilidad del plan de suscripción, disponibilidad del evento según su estado, horario permitido, capacidad disponible y verificación de que el usuario no se encuentre conectado previamente. Finalmente, se genera un registro indicando si el acceso fue autorizado o rechazado junto con su motivo.

Opción 9 – Ver usuarios conectados a un evento

Permite visualizar la lista de usuarios que actualmente se encuentran conectados a un recital en vivo.

Opción 10 – Ver capacidad de un evento

Permite consultar la cantidad de usuarios conectados y compararla con la capacidad máxima establecida para el evento.

Opción 11 – Expulsar usuario de un evento

Permite remover usuarios conectados de eventos que se encuentren en estado **EN_CURSO**, actualizando la lista de participantes del recital.

Opción 12 – Pausar evento

Permite cambiar el estado de un evento que se encuentre en curso al estado **PAUSADO**, impidiendo temporalmente nuevos ingresos.

Opción 13 – Reanudar evento

Permite volver a colocar un evento pausado en estado **EN_CURSO**, permitiendo nuevamente la gestión de accesos.

Opción 14 – Finalizar evento

Permite cambiar el estado de un evento activo o pausado al estado **FINALIZADO**, cerrando definitivamente la posibilidad de nuevos ingresos.

Opción 15 – Ver registros de acceso

Permite consultar el historial de accesos realizados, mostrando tanto ingresos autorizados como intentos rechazados y el motivo asociado a cada caso.

Opción 16 – Guardar datos en archivos TXT

Permite almacenar la información del sistema en archivos de texto mediante la capa de persistencia implementada, conservando los datos generados durante la ejecución.

Opción 17 – Cargar datos desde archivos TXT

Permite recuperar la información previamente almacenada en archivos de texto, restaurando usuarios, artistas y eventos dentro del sistema.

Opción 0 – Salir

Finaliza la ejecución de la aplicación cerrando el menú principal.

Mediante la ejecución de estas opciones se pudieron comprobar los principales casos de uso del módulo, incluyendo la creación y administración de entidades, la gestión del ciclo de vida de los eventos, el control de acceso de usuarios, la generación de registros de trazabilidad y el funcionamiento de la persistencia de datos mediante archivos de texto.

13. Estado final del desarrollo y conclusiones

Actualmente se completó el desarrollo del módulo de acceso a eventos en vivo integrado con la estructura base de UADE Beats. A partir de las observaciones recibidas en la entrega anterior, se realizaron mejoras sobre el diseño y la implementación, incorporando una mejor distribución de responsabilidades entre las clases, la integración de la entidad Artista dentro de los eventos y una mayor participación de GestorEventosEnVivo en la administración de eventos, usuarios, solicitudes de ingreso y registros de acceso.

La versión final del sistema cuenta con la implementación completa de las principales clases del módulo, sus relaciones, atributos, métodos y reglas de negocio correspondientes. Asimismo, se aplicaron conceptos fundamentales de la programación orientada a objetos como herencia, abstracción, encapsulamiento, utilización de colecciones genéricas, enumeraciones y excepciones propias, logrando una solución coherente, modular y alineada con la arquitectura general de UADE Beats.

El módulo implementa funcionalidades relacionadas con:

- validación de solicitudes de ingreso mediante AccesoEvento;
- control de capacidad y administración de usuarios conectados mediante RecitalEnVivo;
- autorización de accesos según el estado del usuario, su plan de suscripción y las restricciones definidas para cada evento;
- administración del ciclo de vida de los eventos mediante EstadoEvento y las reglas implementadas en Evento;
- gestión centralizada de eventos, usuarios, solicitudes de ingreso y registros mediante GestorEventosEnVivo;
- trazabilidad de ingresos autorizados y rechazados mediante RegistroAcceso;
- manejo de errores y rechazo de accesos mediante la excepción propia AccesoDenegadoException;
- integración de artistas asociados a los eventos en vivo mediante la relación entre Evento y Artista.

Además, se realizó la implementación completa del módulo en Java, incluyendo atributos encapsulados, constructores, métodos de negocio, relaciones entre entidades y diferentes interfaces de ejecución que permiten simular los principales casos de uso del sistema tanto mediante una consola interactiva como a través de una interfaz gráfica desarrollada con Swing. Estas interfaces permiten comprobar escenarios como ingresos autorizados, rechazos por falta de permisos, eventos no disponibles, capacidad agotada, usuarios ya conectados y demás validaciones propias del módulo.

Las pruebas funcionales realizadas permitieron verificar el correcto funcionamiento de las reglas de negocio implementadas y la interacción entre las distintas clases del sistema, comprobando que las validaciones y restricciones se ejecutan de acuerdo con los requisitos establecidos.

En conclusión, el trabajo desarrollado permitió obtener un módulo funcional de gestión de acceso a eventos en vivo, consistente con la arquitectura de UADE Beats y con una correcta aplicación de los conceptos y principios de programación orientada a objetos desarrollados durante la cursada.